

AMAZON ML CHALLENGE 2026

Business Entity Resolution

FINAL OPTIMISED PIPELINE

Deep Research Synthesis · 11-Stage Architecture

Blocking · Normalisation · LightGBM · Expected-F0.5 · Exclusivity

11

Pipeline Stages

7

Blocking Legs

~90

Features

≥99%

Target Recall

F0.5

Metric

5 CRITICAL GAPS IDENTIFIED & ADDRESSED

GAP 1: Missing Cross-Script Normalisation (Indic 9-script + French ligatures) | GAP 2: No Exclusivity Constraint | GAP 3: Plain threshold vs Expected-F0.5 | GAP 4: Distractor Density Shift on Leaderboard | GAP 5: No Bidirectional Retrieval

§0 IMPLEMENTATION PRIORITY ORDER

The table below ranks each component by expected impact on the final private leaderboard F0.5. Implement in this exact order — the top three rows are CRITICAL and must be complete before any model training begins. A pipeline with perfect blocking and no ML model outperforms a perfect ML model with leaky blocking.

Priority	Component	Expected Gain	Status in Base
■■■■■ ■	Blocking recall + country partitioning	Recall ceiling $\geq 99\%$	Partial — add bidirectional + key legs
■■■■■ ■	Indic/France cross-script normalisation	+8% recall on 22.7% India cross-script pairs	MISSING — CRITICAL
■■■■■ ■	Hard-negative mining from candidate set	Largest single F0.5 training lift	Mentioned but underdeveloped
■■■■■ ■	Expected-F0.5 subset selection + isotonic calibration	Direct metric optimisation	MISSING — plain threshold only
■■■■■ ■	Exclusivity constraint + singleton handling	Pure precision gain, zero recall cost	MISSING — CRITICAL
■■■■■	Adaptive-K pruner + LightGBM ranker on blocking scores	Smaller, cleaner candidate set	Not present
■■■■■	BGE-M3 / multilingual dense retrieval	Trade-name & transliteration recall	Present but not fine-tuned
■■■■■	Distractor-density-aware features (rank/percentile)	Prevents train/test distribution shift	MISSING — known pitfall
■■■	S2 $\leftrightarrow$ S3 evidence as cross-source feature	Collective consistency signal	Mentioned but not built
■■	Cross-encoder reranker (ONNX int8, CPU)	Covers trade-names, reordered tokens	Optional 2nd stage only

■ KEY INSIGHT

- Do NOT make the embedding model the centrepiece. Make candidate-generation and normalisation the centrepiece.
- A small interpretable LightGBM on well-engineered features beats a large model with poor candidates.
- Biggest gains: (1) not losing pairs in blocking, (2) no density-sensitive false positives, (3) direct F0.5 decision layer.

§1

MULTI-REPRESENTATION NORMALISATION LAYER

Every record must produce **five or more representations** of each field. Do NOT destroy information — retain raw alongside normalised forms so the model can learn that 'ABC Ltd' ↔ 'ABC Limited' is strong evidence while 'ABC' ↔ 'ABC Technologies' is uncertain. All forms are cached to Parquet after generation.

Representation	How Generated	Example	Used By
name_raw	Original text as-is	ABC Pvt. Ltd.	Raw similarity features
name_norm	Unicode NFKD → lowercase → punct strip → whitespace collapse	abc pvt ltd	TF-IDF blocking, Jaro-Winkler
name_core	name_norm minus legal suffixes	abc	Exact key blocking, core match feature
legal_suffix	Extracted legal form token	pvt ltd	Suffix agreement feature
name_tokens	Sorted, de-duped token list	[abc]	Token Jaccard, token-sort ratio
name_transliterated	Indic→Latin (ITRANS, 9 scripts) + French ligature fold	abc privet limted	Cross-script blocking key
address_norm	Abbrev expand + lowercase + digits preserved	21 mg road bengaluru	Address TF-IDF, edit sim
address_numbers	All numeric tokens extracted	[21]	House-number match/conflict feature
address_pin	5–6 digit postal code extracted	560001	PIN/ZIP exact blocking key
is_non_latin	Boolean flag for non-Latin script	True / False	Feature: script-mismatch indicator

Indic Cross-Script Coverage — 9 Script Blocks

22.7% of India S1–S2 pairs are cross-script (Devanagari / Gurmukhi / Bengali ↔ Latin). Without ITRANS-based transliteration, these pairs are **lost before the matcher ever sees them**. One implementation raised Indic candidate recall from 91.3% to 99.4% purely by adding this layer.

Script Block	Unicode Range	Key Pitfall
Devanagari	U+0900–U+097F	Candrabindu marks (U+0900, U+093D) → strip post-transliteration
Bengali	U+0980–U+09FF	Conjunct consonants need schwa deletion
Gurmukhi	U+0A00–U+0A7F	Nukta characters (U+0A3C) → no ITRANS mapping, strip
Gujarati	U+0A80–U+0AFF	Anusvara context-dependent nasalisation
Odia	U+0B00–U+0B7F	Inherent vowel suppression differs from Devanagari
Tamil	U+0B80–U+0BFF	No inherent vowel — all vowels explicit
Telugu	U+0C00–U+0C7F	Length marks on vowels affect transliteration
Kannada	U+0C80–U+0CFF	Shared characters with Telugu — order matters

Malayalam	U+0D00–U+0D7F	Chillu letters need special handling
-----------	---------------	--------------------------------------

French Ligature Folds

œ → oe	æ → ae	ß → ss	ô → o	ñ → n	é/è/ê → e	ç → c	à/â → a
--------	--------	--------	-------	-------	-----------	-------	---------

Learned Normalisation from Training Pairs

Beyond rule-based dictionaries, learn equivalences directly from train_ground_truth.tsv. For every matched pair, align tokens between name_norm(S1) and name_norm(S2/S3). High-frequency mappings (pvt→private, praivet→private, injiniyaring→engineering) become an additional normalisation layer. One pipeline learned **526+ such mappings**. Fully compliant with 'provided data only' rule.

§2

BLOCKING & CANDIDATE GENERATION — 7 LEGS

■ BLOCKING IS THE HARD CEILING

- The blocking recall ceiling is the HARD UPPER BOUND on your F0.5 score.
- If the correct pair is not in candidate_pairs.tsv, no matcher can recover it.
- Target: $\geq 99\%$ blocking recall at ≤ 30 candidates per S1 entity.
- Best practice: 99.78% US / 99.03% India at 24–29 candidates per S1.
- Training ground truth has ZERO cross-country matches → strict country partitioning eliminates 50–70% of search space with zero recall loss.

Country Partitioning — Zero-Cost Recall Preservation

Partition ALL blocking strictly by country. France flows through the same partition logic. **Do NOT hard-code {US, India}** — treat country as an open-set string label as the problem statement requires.

LEG A: Word TF-IDF (name + addr)

- Primary recall workhorse — IDF-weighted token retrieval on normalised name concatenated with address
- Run as sparse TF-IDF with chunked matrix multiplication (avoids OOM on large datasets)
- Top-40 per query → feed to adaptive-K pruner
- Impact: highest-recall single leg; ~85–90% recall alone

LEG B: Character 4-gram TF-IDF on name

- Character n-grams ($n=3-5$) on name_norm with spaces removed
- Handles abbreviations, typos, transliteration variants — e.g. 'microsof' still matches 'microsoft'
- Runs on name only (not address) to avoid noise from varied address formats

LEG C: Bidirectional TF-IDF (CRITICAL — often missing)

- Run gallery (S2/S3 records) as queries against the S1 index → union with forward pass
- Recovers rare-name entities where S2/S3 side vocabulary is more discriminative than S1
- Adds +1–2% recall at near-zero cost — simply reuse the same TF-IDF index

LEG D: Exact / Near-Exact Key Blocking

- 7 key types: sorted-token signature, name-core + house-no, unique-name-only, name+house-no, acronym, no-space name+no, exact address ≥ 12 chars
- Bucket cap = 200 entries; take top-k=30 per bucket to avoid mega-clusters
- Catches perfect or near-perfect matches that TF-IDF may deprioritise due to IDF weighting

LEG E: PIN / ZIP Code Blocking

- Extract all 5–6 digit postal codes from address fields
- Bucket all S2/S3 records sharing the same PIN/ZIP with each S1 record
- Cap per bucket to avoid mega-clusters in dense postal areas
- High precision: same postal code + partial name match → very likely true positive

LEG F: Phonetic / Initialism Blocking

- Soundex / Metaphone on name_core tokens for phonetic variant recall
- Initialism keys: first-letter sequences (e.g. 'IBM' ↔ 'International Business Machines')
- Bucket cap = 200 per code; skip oversized buckets (reported: 50M+ candidates at full scale on common Soundex codes)

LEG G: Dense Embedding ANN — BGE-M3 / multilingual-e5

- Encode name_norm as embedding; FAISS HNSW index or sign-code Hamming shortlist + INT8 rescore
- Top-40 per query; handle trade-names and reordered tokens that lexical methods miss
- Fine-tune encoder on training pairs (contrastive InfoNCE + Matryoshka loss) for significant recall improvement over zero-shot
- Model must be ≤8B parameters and MIT/Apache 2.0 licensed

Candidate Count vs Recall Trade-off

K (candidates/S1)	Blocking Recall (US)	Blocking Recall (India)	Notes
≤10	~95%	~92%	Too aggressive — measure on your own data
≤20	~98%	~96%	Good balance
≤30 ✓ TARGET	99.78%	99.03%	Reported by top implementation (SABER)
≤50	~99.9%	~99.7%	Diminishing returns — penalises reduction ratio

Adaptive-K Pruner → candidate_pairs.tsv

After unioning all 7 legs, apply an **Adaptive-K pruner** rather than a fixed top-K. Keep a candidate if: rank < k_min **OR** score ≥ (top1_score – gap). Then run a fast LightGBM ranker on cheap features (blocker scores, name edit ratio, address token overlap, candidate source indicators) to re-rank and prune to ≤30 per S1. This final set is what goes into **candidate_pairs.tsv** — the file Amazon judges for reduction ratio.

Candidate Source Indicator Features (Surprise Feature)

- Give LightGBM binary flags for which legs retrieved each pair: name_block_hit, address_block_hit, tfidf_block_hit, embedding_block_hit, phonetic_block_hit, number_block_hit, bidirectional_block_hit
- A pair retrieved by 4+ legs with consistent evidence is far safer than one retrieved only by embedding
- num_legs_retrieved (0–7, continuous) is often the single most predictive blocking feature

§3

FEATURE ENGINEERING — ~90 FEATURES IN 6 GROUPS

Features fall into six groups. The most important are the **cross-field interaction** and **evidence-conflict** features — they separate a strong model from a weak one for the F0.5 objective. Critical warning: **NEVER use raw candidate counts** as features (`n_cand`, `g_size`, `cvc_*`) — they are density-sensitive and will cause validation F0.5 to diverge from leaderboard F0.5.

Name Similarity Features (18 features)

- Levenshtein ratio (rapidfuzz partial)
- Jaro-Winkler similarity
- Character TF-IDF cosine (IDF from training corpus)
- Token Jaccard
- Token overlap (one-sided containment)
- Token-sort ratio (sort tokens then edit sim)
- Exact normalised match (bool)
- Exact name_core match (bool)
- Phonetic similarity (Soundex/Metaphone codes match)
- Prefix word overlap (first 2 tokens match)
- Name length difference (abs)
- Number of distinct tokens
- Legal suffix exact match
- Legal suffix both-present flag
- Name frequency in pool (log-transformed)
- Name as single token edit sim (no spaces)
- Sorted-token edit sim
- Trigram Jaccard

Address Similarity Features (16 features)

- Levenshtein ratio
- Jaro-Winkler similarity
- Character cosine (3-gram)
- Token Jaccard
- Token overlap
- House-number exact match (bool)
- House-number conflict (bool) ← HIGH PRECISION SIGNAL
- PIN/ZIP exact match (bool) ← HIGH PRECISION SIGNAL
- City token overlap
- Street token overlap
- Numeric token overlap
- Address length difference
- Both-have-address flag
- Address token count difference
- State/region token match
- Landmark token overlap

Cross-Field Interaction Features (10 features)

- $\text{name_sim} \times \text{address_sim}$ (product) ← MOST IMPORTANT
- $\text{name_sim} + \text{address_sim}$ (sum)
- $\max(\text{name_sim}, \text{address_sim})$
- $\min(\text{name_sim}, \text{address_sim})$
- $\text{name_high_address_low}$: $\text{name_sim} > 0.90$ AND $\text{address_sim} < 0.20$
- $\text{name_low_address_high}$: $\text{name_sim} < 0.50$ AND $\text{address_sim} > 0.80$
- $\text{name_address_agree}$: both > 0.75 (bool)
- numeric_conflict : house numbers differ (bool) ← USE AS HARD VETO TOO
- $\text{cross_field_token_overlap}$: name tokens appear in address
- $\text{country_exact_match}$ (bool)

Semantic / Embedding Features (4 features)

- BGE-M3 or multilingual-e5 cosine similarity
- Embedding similarity $\times$ name_sim (interaction)
- Embedding rank among all S1 candidates (density-safe)
- Score gap from best candidate (margin feature)

Competition / Context Features — DENSITY-SAFE (12 features)

- Rank of this candidate among all S1's candidates
- Score gap from top-1 candidate (relative)
- Score percentile within S1's candidate set
- Number of S1 candidates with sim > 0.9 (ambiguity count)
- Candidate name frequency rank in pool
- S2↔S3 agreement score (if S2 and S3 candidate near-duplicate → feature)
- Source pair: S1-S2 vs S1-S3 (binary flag)
- IDF of candidate name (from per-country TF-IDF)
- Name uniqueness in pool (1 / frequency)
- Runner-up score gap (precision guard)
- ■ AVOID: raw counts (n_cand, g_size, cvc_*) — density-sensitive!
- ■ AVOID: absolute pool sizes — use rank/percentile ALWAYS

Blocking Evidence Features (7+ features)

- name_tfidf_block_hit (bool)
- char_ngram_block_hit (bool)
- address_block_hit (bool)
- exact_key_hit (bool)
- phonetic_block_hit (bool)
- embedding_block_hit (bool)
- bidirectional_block_hit (bool)
- num_legs_retrieved (0–7, continuous) ← often strongest single feature

§4 TRAINING STRATEGY & NEGATIVE MINING

The training strategy has two phases. Random negatives are easy — the model trivially learns to separate them. **Hard negatives from the candidate set** are what determine leaderboard performance.

Step 1: Entity-Aware Train/Val Split

- Split at the S1 entity level — NEVER at the pair level
- All candidate pairs for a given S1 entity must stay in the same fold
- Leaking entity information across folds produces optimistic validation that does not predict leaderboard performance
- Also create leave-one-country-out splits: train on India, validate on US and vice versa
- This simulates the unseen-France generalisation scenario

Step 2: Phase 1 — Warm-up with Random + Easy Hard Negatives

- Train initial LightGBM on: (a) all true positive pairs from training GT, (b) random negatives from candidate set
- This gives a warm-start model that can identify medium-difficulty cases
- Use AUC as training metric at this stage

Step 3: Phase 2 — Hard Negative Mining (HIGHEST IMPACT)

- Run Phase 1 model on all candidates
- Extract hard negatives: non-match candidates with $P(\text{match}) \geq 0.3$ from Phase 1
- Add to training set: true positives + hard negatives + random negatives at 2:1:1 ratio
- Re-train LightGBM on combined set — this is the single highest-impact training improvement

Step 4: Isotonic Calibration

- Fit `sklearn.calibration.CalibratedClassifierCV (method='isotonic')` on validation fold OOF predictions
- Calibration makes `model.predict_proba()` output approximate true $P(\text{match})$
- Required for the expected-F0.5 decision layer to work correctly
- Validate with reliability diagram (calibration plot) before proceeding to decision layer

Step 5: Distractor Density Matching

- Test set has ~2x more distractor records per S1 than training (5.8 vs 4.67 pool records per S1)
- Simulate by upweighting hard negatives ~2x in training
- OR artificially increase candidate pool density in validation fold
- Remove or rank-normalise any feature that is an absolute count of candidates
- One team's validation F0.5 went UP while leaderboard score went DOWN due to this exact shift

LightGBM Hyperparameters

n_estimators	1000+	num_leaves	63+
learning_rate	0.05	min_child_samples	20
subsample	0.8	colsample_bytree	0.8
reg_alpha	0.1	reg_lambda	0.1

early_stopping_rounds	50	metric	auc (train) → F0.5-tune threshold post-calibration
-----------------------	----	--------	---

§5

F0.5-OPTIMISED DECISION LAYER

Expected-F0.5 Subset Selection (Closed Form)

For each S1 entity, given candidates with calibrated scores $p_1 \geq p_2 \geq \dots \geq p_n$, the expected F0.5 for accepting the top-k subset has a closed form:

$$E[F_{0.5}(k)] = 1.25 \times \sum p_i(i \leq k) / (k + 0.25 \times \sum p_i(i \leq k))$$

Evaluate for $k = 0, 1, \dots, n$ and pick the k that maximises this value. $k=0$ (predict singleton) scores 1.0 if the entity truly has no matches — the model **automatically learns when to predict empty**.

1	Run LightGBM.predict_proba() on all candidate pairs
2	Apply isotonic calibration to get true P(match) scores
3	For each S1 entity: sort candidates by P descending
4	Evaluate expected-F0.5(k) for $k = 0 \dots n$; select argmax k
5	Apply exclusivity: if a S2/S3 candidate was already claimed by a higher-probability S1, remove and re-evaluate
6	Hard veto: if house_number_conflict AND address_sim < 0.3 → reject regardless of P
7	Hard veto: if name_sim < 0.3 AND embedding_sim < 0.4 → reject
8	For uncertain pairs ($0.3 \leq P \leq 0.6$): run cross-encoder reranker if enabled
9	Write accepted pairs to matching_results.tsv; write all candidates to candidate_pairs.tsv

Singleton Handling — Do Not Neglect

A Source 1 entity with no true matches scores **1.0** when predicted empty and **0.0** for any prediction. 25–27% of S2/S3 entities are non-matching distractors. Correctly predicting singletons earns significant credit. The expected-F0.5 framework handles singletons automatically: when $\max(P)$ is low, $k=0$ gives the best expected score.

Additionally train an entity-level 'has_any_match' classifier on entity-level aggregates: max candidate P, number of candidates above threshold, name_frequency. Use its output as a second signal on singleton prediction.

Exclusivity Constraint — Implementation

■ VERIFY BEFORE APPLYING

- Run: assert no repeated S2-/S3- IDs across S1 rows in train_ground_truth.tsv
- Training GT has ZERO cases of one S2/S3 record matching multiple S1 entities
- If assertion passes: enforce exclusivity via greedy highest-probability assignment
- For each S2/S3 that appears in multiple S1 match sets: assign to S1 with highest P(match), remove from all others
- This is a PURE PRECISION GAIN — costs nothing in recall

§6

COMPLETE 11-STAGE PIPELINE ARCHITECTURE

Each stage must complete and be verified before the next begins. Stages 3–4 form a loop: do not proceed to Stage 5 until blocking recall is $\geq 99\%$.

1

Load & Validate Data

- Read `train_source{1,2,3}.tsv` and `test_source{1,2,3}.tsv` with `sep='\t'` (CRITICAL — never read without it)
- Verify no duplicate `entity_ids` within each source file
- Check all ground truth S1 IDs exist in `source1`
- EDA: match distribution, country distribution, distractor density per S1
- Assert: 0 cross-country matches in ground truth (enables country partitioning)
- Assert: no repeated S2-/S3- IDs across S1 rows (enables exclusivity)

2

Multi-Representation Normalisation

- Unicode NFKD normalisation + lowercase
- Indic→Latin transliteration (9 scripts, ITRANS + schwa deletion)
- French ligature fold ($\text{œ} \rightarrow \text{oe}$, $\text{æ} \rightarrow \text{ae}$, $\text{ß} \rightarrow \text{ss}$, $\text{ô} \rightarrow \text{o}$, $\text{ñ} \rightarrow \text{n}$)
- Legal suffix extraction (pvt, ltd, corp, sarl, inc, llc, gmbh, ...)
- Address abbreviation expansion (Rd→Road, St→Street, Blvd→Boulevard, ...)
- Learned normalisation from training pairs (526+ mappings)
- Extract: numbers, PIN/ZIP, city tokens, street tokens
- Cache normalised forms as Parquet for fast re-use

3

Candidate Generation — 7 Legs, Country-Partitioned

- Leg A: Word TF-IDF (name+addr), chunked sparse, top-40
- Leg B: Character 4-gram TF-IDF on name (spaces removed), top-40
- Leg C: Bidirectional TF-IDF — gallery as queries → union with forward
- Leg D: Exact/near-exact key blocking (7 key types, bucket cap=200)
- Leg E: PIN/ZIP blocking (5–6 digit match)
- Leg F: Phonetic/initialism blocking (Soundex, skip buckets > 200)
- Leg G: Dense embedding ANN — BGE-M3 or fine-tuned multilingual-e5, FAISS HNSW
- UNION all legs → raw candidate set

4

Blocking Recall Diagnostics (GATE — DO NOT SKIP)

- Compute blocking recall on training fold BEFORE proceeding
- Target: $\geq 99\%$ recall, ≤ 30 candidates per S1
- Report: per-country recall, avg candidates, reduction ratio
- Identify missed positives — are they cross-script? address-only? rare names?
- DO NOT proceed to Stage 5 until recall is satisfactory

5

Adaptive-K Pruning → candidate_pairs.tsv

- Run LightGBM ranker on cheap features (blocker scores + 5–10 fast similarities)
- Adaptive-K: keep rank < k_min OR score ≥ (top1 – gap), cap at 30
- This is the EXACT set that goes into candidate_pairs.tsv
- Verify: every true match from training is still present → recall check

6

Full Pairwise Feature Extraction (~90 features)

- Compute all 6 feature groups (name, address, cross-field, semantic, competition, blocking)
- CRITICAL: use rank/percentile for competition features, NOT raw counts
- S2↔S3 agreement: check if top S2 and S3 candidates for same S1 are near-duplicates
- Cache feature matrix to Parquet

7

Entity-Aware Train/Val Split

- Split at S1 entity level (80/20)
- Create: US-only-val, India-only-val splits for country generalisation test
- Stratify: ensure singletons and multi-match entities both appear in each fold

8

Two-Phase LightGBM Training

- Phase 1: true positives + random negatives from candidate set
- Phase 2: add hard negatives (non-matches with Phase 1 score ≥ 0.3)
- Upweight hard negatives ~2x to simulate test distractor density
- Hyperparams: n_estimators=1000+, learning_rate=0.05, num_leaves=63+, early stopping on AUC

9

Isotonic Calibration

- Fit IsotonicRegression (sklearn) on validation fold OOF predictions
- Calibration makes scores true probabilities for expected-F0.5 selection
- Validate calibration plot (reliability diagram) before proceeding

10

F0.5-Optimised Decision Layer

- Per-entity: expected-F0.5 subset selection (closed-form argmax over k)
- Apply exclusivity: each S2/S3 claimed by highest-P S1 only
- Apply hard vetoes: house-number conflict + low address sim → reject
- Optional: cross-encoder reranker on uncertain pairs ($P \in [0.3, 0.6]$)
- Singleton: if k=0 selected, write empty matched_entity_ids

11

Consistency Checks + Output

- S2↔S3 evidence: if S2-X and S3-Y are near-duplicates and both match S1-A, add extra confidence
- Do NOT do unrestricted transitive merging (false S1-S2 drags unrelated S3 in)
- Write `matching_results.tsv` and `candidate_pairs.tsv`
- Run official validator: `python3 utils/validate_submission.py ...`
- Run F0.5 scorer on validation fold: confirm macro F0.5 matches expectations

§7 PROPOSED vs OPTIMISED — DELTA TABLE

Use this table to audit any existing pipeline against the optimised architecture. Each row is a component; the Status column shows what action is needed.

Component	Proposed	Optimised (This Document)	Impact
Normalisation	lowercase + punct + suffix strip	5-layer multi-repr + 9-script Indic transliteration + France ligatures + learned mappings	+8% recall on 22.7% cross-script India pairs
Blocking legs	3 legs: token+phonetic, TF-IDF, BGE-M3	7 legs incl. bidirectional, 4-gram char, exact keys, PIN, adaptive-K prune + ranker	99%+ recall at ≤30 candidates
Country handling	Generic — not partitioned	Strict country partitioning (eliminates 50–70% search space with 0 recall loss)	2–5x faster blocking, better precision
Negatives	Random negatives only	2-phase: random then hard-negative mining from candidate set, density-matched	Largest single training improvement
Decision layer	Single global threshold	Per-entity expected-F0.5 selection (closed-form) + isotonic calibration	Direct metric optimisation
Exclusivity	Not present	Greedy highest-P assignment, verified against ground truth	Pure precision gain, 0 recall cost
Competition features	Not specified / raw counts	Rank/percentile based, NOT raw counts — density-agnostic	Prevents train/test distribution shift
S2↔S3 evidence	Mentioned but not built	Near-duplicate score as additional feature (no transitive merge)	Collective consistency signal
Validation	Entity holdout	Entity holdout + leave-one-country-out + distractor density stress-test (2x pool)	Catches train/test gap before submission
Cross-encoder	Bake-off vs LightGBM	Optional 2nd stage on uncertain pairs only ($P \in [0.3, 0.6]$); ONNX int8 CPU	Cost-effective; not mandatory

§8 OUTPUT FORMAT & SUBMISSION VALIDATION

File	Column	Example Value	Notes
matching_results.tsv	source1_entity_id	S1-00001	Entity_id of Source 1 record
matching_results.tsv	matched_entity_ids	S2-00047,S3-00812	Comma-sep S2/S3 IDs; empty for singletons
candidate_pairs.tsv	source1_entity_id	S1-00001	Same as above
candidate_pairs.tsv	candidate_entity_ids	S2-00047,S3-00812,S3-00999	ALL candidates fed to model (superset of matches)

Validation Checklist

- Every Source 1 entity in test set has exactly one row in both output files
- `matched_entity_ids` are a subset of `candidate_entity_ids` (pipeline bug otherwise)
- No duplicate entity IDs within any single ID list
- No duplicate `source1_entity_id` rows
- Only S2-/S3- IDs that exist in the test set; no S1- self-matches
- Run: `python3 utils/validate_submission.py --matching output/matching_results.tsv --candidate output/candidate_pairs.tsv --test-dir dataset/test`
- Expect PASS (exit 0) before uploading to leaderboard portal

■ F0.5 FORMULA REMINDER

- $F_{0.5} = (1.25 \times \text{Precision} \times \text{Recall}) / (0.25 \times \text{Precision} + \text{Recall})$
- Computed as MACRO-AVERAGE: F0.5 is calculated per Source 1 entity, then averaged across all S1 entities
- Singletons INCLUDED: correct empty prediction = 1.0; any false match prediction = 0.0
- Precision-heavy: F0.5 weights precision 2× over recall — avoid false merges above all else

§9

COMMON PITFALLS & HOW TO AVOID THEM

Pitfall 1: Reading TSV without sep='t'

Silently produces a single column containing the whole line. Always pass `sep='t'` to `pd.read_csv()`.

Pitfall 2: Hard-coding country filter to {US, India}

Test set has France. Country must be an open-set string label throughout the pipeline. Never hard-code.

Pitfall 3: Cross-script pairs silently dropped in blocking

Indic script records never surface as candidates without ITRANS transliteration. Add transliteration BEFORE blocking, not after.

Pitfall 4: Density-sensitive features causing train/test distribution shift

Raw counts (`n_cand`, `g_size`) differ between train and test (5.8 vs 4.67 pool records per S1). Use rank/percentile.

Pitfall 5: Pair-level train/val split

Leaks entity information across folds. Always split at S1 entity level.

Pitfall 6: Single global threshold instead of per-entity expected-F0.5

F0.5 is computed per entity before macro-averaging. A global threshold is provably suboptimal.

Pitfall 7: Not enforcing exclusivity

If one S2/S3 ID appears in multiple S1 match predictions, this is almost certainly wrong. Verify against GT and enforce.

Pitfall 8: Forgetting singletons in training

25–27% of entities have no match. If the model never sees singleton examples, it over-predicts matches.

Pitfall 9: Unrestricted transitive merging

Do NOT merge $S1-A \rightarrow S2-X$ and $S1-A \rightarrow S3-Y$ and then assume $S2-X \rightarrow S3-Y$ is evidence. Only use $S2 \leftrightarrow S3$ near-duplicate as a feature.

Pitfall 10: Using external APIs / databases

Strictly prohibited. Disqualification risk. All entity resolution must use only provided training data.

§10 QUICK REFERENCE — KEY CODE PATTERNS

Read data correctly

```
import pandas as pd

s1 = pd.read_csv("dataset/train/train_source1.tsv", sep="\t")
s2 = pd.read_csv("dataset/train/train_source2.tsv", sep="\t")
gt = pd.read_csv("dataset/train/train_ground_truth.tsv", sep="\t")

# Verify no cross-country matches
merged = s1[["entity_id", "country"]].merge(
    gt.explode("matched_entity_ids"), left_on="entity_id", right_on="source1_entity_id")
assert (merged.groupby("entity_id")["country"].nunique() == 1).all()
```

Expected-F0.5 subset selection

```
import numpy as np

def expected_f05_select(probs):
    """Return top-k indices maximising expected F0.5."""
    probs = np.sort(probs)[::-1] # descending
    best_k, best_ef05 = 0, 0.0
    cumsum = 0.0
    for k, p in enumerate(probs, 1):
        cumsum += p
        ef05 = 1.25 * cumsum / (k + 0.25 * cumsum)
        if ef05 > best_ef05:
            best_ef05, best_k = ef05, k
    return best_k # 0 = predict singleton
```

Isotonic calibration

```
from sklearn.isotonic import IsotonicRegression
import numpy as np

cal = IsotonicRegression(out_of_bounds="clip")
# oof_probs, oof_labels from validation fold
cal.fit(oof_probs, oof_labels)
calibrated_probs = cal.predict(test_probs)
```

Exclusivity enforcement

```
def enforce_exclusivity(matches_dict):
    """matches_dict: {s1_id: [(s2s3_id, prob), ...]}"""
    claimed = {} # s2s3_id -> (s1_id, best_prob)
    result = {s1: [] for s1 in matches_dict}
    # Sort all pairs by prob descending
    all_pairs = [(p, s1, s23) for s1, pairs in matches_dict.items()
                  for s23, p in pairs]
    for prob, s1, s23 in sorted(all_pairs, reverse=True):
        if s23 not in claimed:
            claimed[s23] = s1
```

```
        result[s1].append(s23)
    return result
```

Adaptive-K pruner

```
def adaptive_k_prune(candidates, scores, k_min=5, gap=0.1, k_max=30):
    """Keep rank < k_min OR score >= top1 - gap, cap at k_max."""
    if not candidates:
        return []
    top1 = max(scores)
    threshold = top1 - gap
    kept = [(c, s) for c, s in zip(candidates, scores)
            if s >= threshold]
    kept.sort(key=lambda x: -x[1])
    # Ensure at least k_min are kept
    full = sorted(zip(candidates, scores), key=lambda x: -x[1])
    if len(kept) < k_min:
        kept = full[:k_min]
    return [c for c, _ in kept[:k_max]]
```

§11 FINAL CHECKLIST BEFORE SUBMISSION

Run through every item. A checked item means it is complete and verified.

NORMALISATION

☒	Unicode NFKD + lowercase applied to all fields
☒	Indic transliteration (9 scripts) implemented and tested
☒	French ligature folds applied
☒	Legal suffix extraction working
☒	Address abbreviation expansion applied
☒	Learned normalisation mappings trained from GT pairs

BLOCKING

☒	All 7 legs implemented and producing candidates
☒	Country partitioning enforced (no cross-country candidates)
☒	Bidirectional retrieval (Leg C) implemented
☒	Blocking recall measured: $\geq 99\%$ US, $\geq 99\%$ India on training fold
☒	Average candidates per S1: ≤ 30
☒	Adaptive-K pruner applied → candidate_pairs.tsv finalised

FEATURES & TRAINING

☒	All ~90 features computed (6 groups)
☒	Competition features are rank/percentile — NOT raw counts
☒	Entity-aware train/val split (S1 level, not pair level)
☒	Phase 1 training complete (true pos + random neg)
☒	Hard negatives mined (Phase 1 score ≥ 0.3 non-matches)
☒	Phase 2 training complete with 2:1:1 ratio
☒	Isotonic calibration fitted and validated (reliability diagram)

DECISION LAYER

☒	Expected-F0.5 subset selection implemented per entity
☒	Exclusivity assertion verified against training GT
☒	Exclusivity enforcement applied to test predictions
☒	Hard vetoes applied (house-number conflict, low name+embedding sim)
☒	Singleton prediction working correctly

OUTPUT & VALIDATION

■	matching_results.tsv: one row per S1 test entity
■	candidate_pairs.tsv: all candidates (superset of matches)
■	All matched IDs are subsets of candidate IDs
■	No duplicate IDs within any list, no duplicate S1 rows
■	python3 utils/validate_submission.py → PASS
■	Validation fold macro F0.5 computed and sensible

■ FINAL REMINDER

- F0.5 penalises false positives 2x more than false negatives. When in doubt, predict singleton.
- The leaderboard scores only matching_results.tsv. Amazon also reviews candidate_pairs.tsv for reduction ratio.
- A smaller, higher-recall candidate set beats a large, low-precision one.
- External data lookup (APIs, geocoding, databases) = DISQUALIFICATION.